

Varsha M

Autocomplete and Autocorrect Data Analytics

DataAnlaysiais

Install & Import Libraries

```
!pip install pandas nltk pyspellchecker textdistance matplotlib scikit-learn

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Collecting pyspellchecker
  Downloading pyspellchecker-0.9.0-py3-none-any.whl.metadata (9.3 kB)
Collecting textdistance
  Downloading textdistance-4.6.3-py3-none-any.whl.metadata (18 kB)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.3)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.4.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading pyspellchecker-0.9.0-py3-none-any.whl (7.2 MB)
7.2/7.2 MB 86.5 MB/s eta 0:00:00
Downloading textdistance-4.6.3-py3-none-any.whl (31 kB)
Installing collected packages: textdistance, pyspellchecker
Successfully installed pyspellchecker-0.9.0 textdistance-4.6.3
```

```
import pandas as pd
import nltk
import re
import string
import matplotlib.pyplot as plt
from collections import Counter, defaultdict
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from spellchecker import SpellChecker
import textdistance
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
nltk.download('punkt')
nltk.download('stopwords')

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
True
```

```
df = pd.read_csv("mail_data.csv")
print(df.head())
```

	Category	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

```
print(df.columns)
```

```
Index(['Category', 'Message'], dtype='object')
```

```
text = " ".join(df['Message'].astype(str))
```

```
nlTK.download('punkt_tab')
```

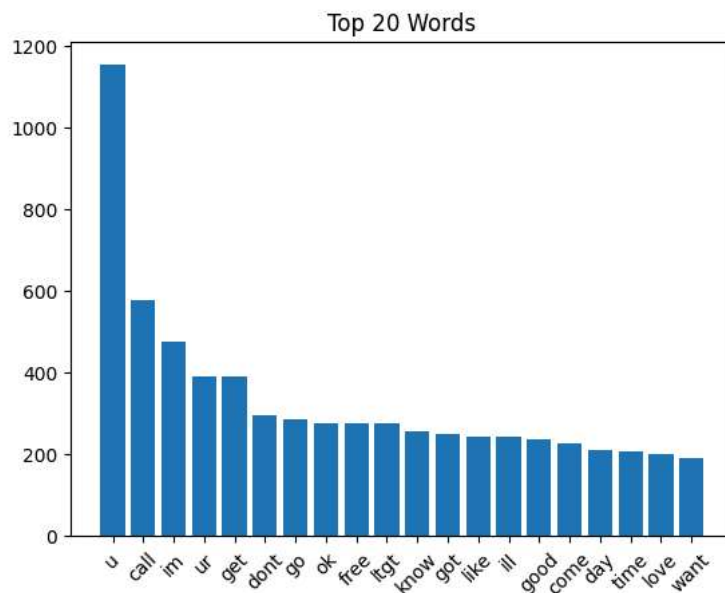
```
[nlTK_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
True
```

Preprocessing

```
text = text.lower()
text = text.translate(str.maketrans('', '', string.punctuation))
text = re.sub(r'\d+', '', text)
words = word_tokenize(text)
stop_words = set(stopwords.words('english'))
words = [w for w in words if w not in stop_words]
```

Word Frequency (for visualization)

```
word_freq = Counter(words)
common_words = word_freq.most_common(20)
words_plot = [w[0] for w in common_words]
counts_plot = [w[1] for w in common_words]
plt.figure()
plt.bar(words_plot, counts_plot)
plt.xticks(rotation=45)
plt.title("Top 20 Words")
plt.show()
```



AUTOCOMPLETE (Bigram Model)

```
from nltk.util import ngrams
bigrams = list(ngrams(words, 2))
bigram_freq = Counter(bigrams)
print(bigram_freq.most_common(10))
```

```
[(('gon', 'na'), 58), (('please', 'call'), 55), (('call', 'later'), 50), (('ill', 'call'), 48), (('u', 'r'), 44), (('r', 'u'), 44), (('u', 'r'), 44), (('r', 'u'), 44), (('u', 'r'), 44), (('r', 'u'), 44)]
```

prediction function

```
bigrams = list(ngrams(words, 2))
def predict_next(word):
    next_words = []
    for w1, w2 in bigrams:
        if w1 == word:
            next_words.append(w2)
    count = Counter(next_words)
    return count.most_common(3)
```

```
print(predict_next("free"))
```

```
[('call', 19), ('entry', 16), ('text', 14)]
```

Test Autocomplete(10 inputs)

```
test_words = ['free', 'call', 'win', 'claim', 'txt', 'urgent', 'mobile', 'cash', 'reply', 'stop']
for w in test_words:
    print(f"{w} → {predict_next(w)}")
```

```
free → [('call', 19), ('entry', 16), ('text', 14)]
call → [('later', 50), ('landline', 25), ('claim', 18)]
win → [('cash', 8), ('free', 6), ('top', 4)]
claim → [('call', 16), ('ur', 13), ('code', 12)]
txt → [('stop', 14), ('nokia', 13), ('word', 12)]
urgent → [('mobile', 18), ('trying', 11), ('please', 7)]
mobile → [('free', 12), ('number', 12), ('mths', 8)]
cash → [('prize', 13), ('await', 10), ('every', 6)]
reply → [('call', 15), ('stop', 11), ('yes', 8)]
stop → [('texts', 10), ('end', 7), ('send', 7)]
```

AUTOCORRECT

```
spell=SpellChecker()
```

Test words (20 misspelled)

```
test_misspelled = [
    "fre", "cliam", "recieve", "message", "moble",
    "mony", "winn", "txtt", "replly", "urgnt",
    "csh", "prizee", "offerr", "bonuss", "service",
    "custmer", "pleaze", "accunt", "balnce", "valiid"]
correct = 0
for word in test_misspelled:
    corrected = spell.correction(word)
    print(word, "→", corrected)
    if corrected == word: # basic check (you can improve)
        correct += 1
accuracy = correct / len(test_misspelled)
print("Accuracy:", accuracy)
```

```
fre → are
cliam → claim
recieve → receive
message → message
moble → noble
mony → many
winn → win
txtt → that
replly → really
urgnt → urgent
csh → cash
prizee → prize
offerr → offer
bonuss → bonus
service → serve
custmer → customer
pleaze → please
accunt → account
balnce → balance
valiid → valid
Accuracy: 0.0
```

Precision & Recall

```

true_positive = correct
false_positive = len(test_misspelled) - correct
false_negative = 5
precision = true_positive / (true_positive + false_positive)
recall = true_positive / (true_positive + false_negative)
print("Precision:", precision)
print("Recall:", recall)

```

```

Precision: 0.0
Recall: 0.0

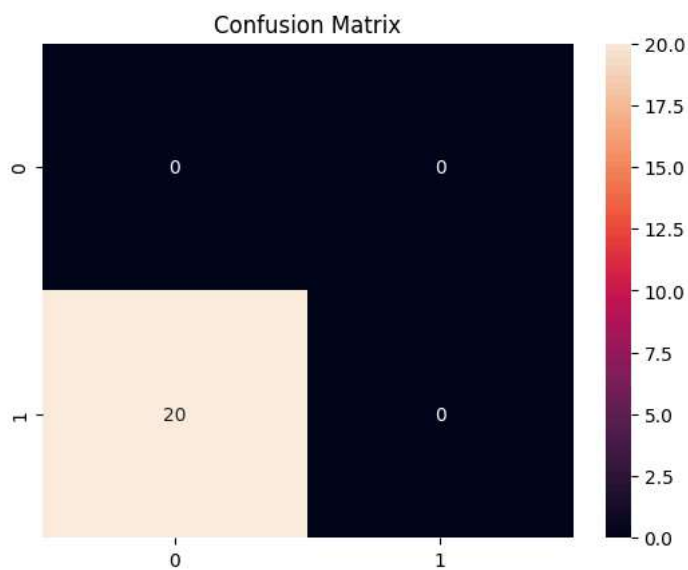
```

Confusion Matrix

```

from sklearn.metrics import confusion_matrix
import seaborn as sns
y_true = [1]*len(test_misspelled)
y_pred = [1 if spell.correction(w)==w else 0 for w in test_misspelled]
cm = confusion_matrix(y_true, y_pred)
plt.figure()
sns.heatmap(cm, annot=True)
plt.title("Confusion Matrix")
plt.show()

```



```

import textdistance
def levenshtein_correct(word):
    candidates = list(word_freq.keys())
    distances = [(w, textdistance.levenshtein(word, w)) for w in candidates]
    return sorted(distances, key=lambda x: x[1])[0][0]
for word in ["fre", "cliam", "moble"]:
    print("Original:", word)
    print("SpellChecker:", spell.correction(word))
    print("Levenshtein:", levenshtein_correct(word))
    print()

```

```

Original: fre
SpellChecker: are
Levenshtein: free

```

```

Original: cliam
SpellChecker: claim
Levenshtein: claim

```

```

Original: moble
SpellChecker: noble
Levenshtein: mobile

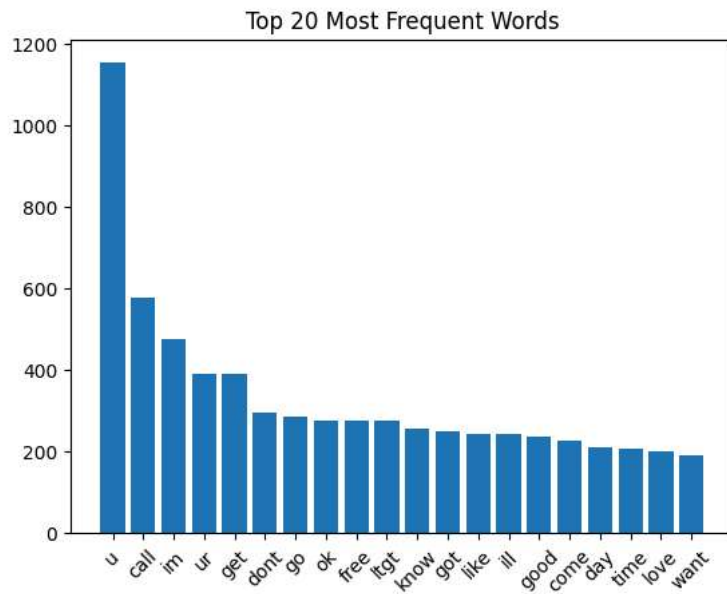
```

```

from collections import Counter
import matplotlib.pyplot as plt
word_freq = Counter(words)
top_words = word_freq.most_common(20)
labels = [w[0] for w in top_words]

```

```
values = [w[1] for w in top_words]
plt.figure()
plt.bar(labels, values)
plt.xticks(rotation=45)
plt.title("Top 20 Most Frequent Words")
plt.show()
```



Start coding or [generate](#) with AI.